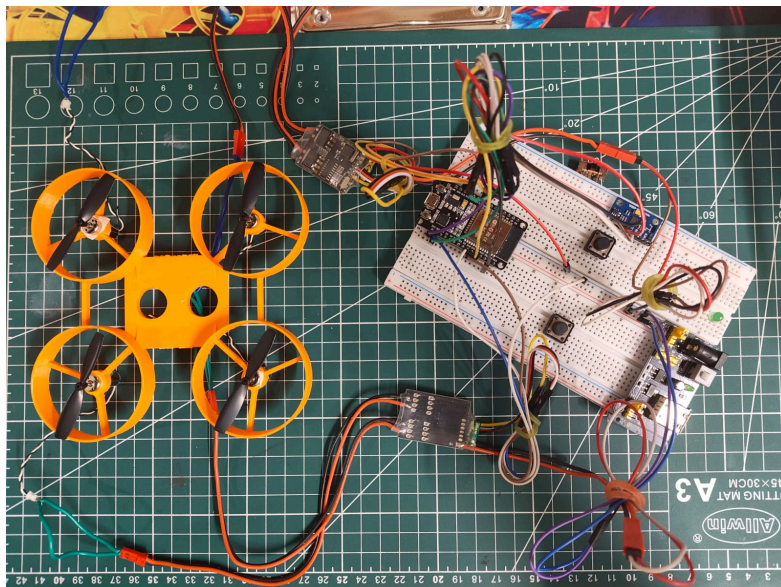
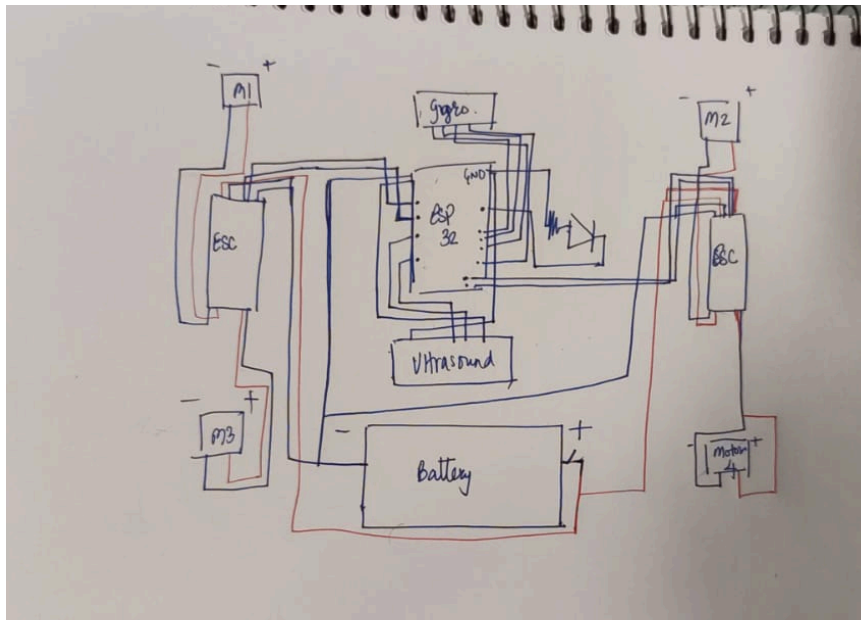


Electronics Documentation

1. Electronics Block Diagram

Show signal and power flow



2. Components Table

Component	Quantity	Purpose	Justification
ESP32	1	To serve as main microcontroller	Multiple functions like bluetooth, motors and PWM required
Dual ESC brushed	2	To serve as flight controllers	Regulates speed of brushed motors and distributes power evenly
Coreless brushed Motors	4	To drive propellers	Coreless motors are the lightest and most suitable for a small drone
propellers	4	To create thrust	Four propellers create even thrust using bernoullis theorem, carrying more central mass and having more control over the pitch yaw and roll
MPU6500	1	Serve as an IMU	Accelerometer and Gyroscope needed for drone to maintain stable flight.
HCS04	1	Detect distance of surface below drone	Ultrasonic sensors work better than IR sensors or motion sensors to detect a hand or a surface under the drone.

Justification is important

Contents of the Folder

1. Electronic Details Document

2. Circuit Diagram

 **Guidelines** Ensure diagrams are neat and readable Can be hand-drawn or software-generated Should match actual implementation Justification of components is mandatory

Fabrication and Integration Process

1. Material Choice Justification :

Why choose Acrylic over MDF? Why PLA instead of Resin? (e.g., weight, durability, aesthetic)

We 3D printed the drone frame out of resin because it was where we had the most control over the density of the product. Since the frame has to be very light, we had to make sure it was crisp and did not have any extra joinery making it heavier.

2. Fabrication Steps :

A documented "Build-log" (e.g., "1. Laser cutting frame," "2. Heat-staking threaded inserts").

- 1) Sketched out the component sizes, especially the motors*
- 2) Downloaded an existing drone frame .stl that matched the motor sizes to save time on measurements*
- 3) Modified the center of the frame to accommodate project specific components or Rhino and Blender*
- 4) Added our initials to the side*
- 5) 3D printed the frame overnight*
- 6) Refined the frame and removed residue and supports*
- 7) Perfectly fit the HCS04 and motors, so we directly mounted those components.*

There was only 1 evolution where the middle was pushed down to create more vertical space in the model.

Contents of the folder

- 1. Drawings & Sketches: Initial ideas, Concept sketches, Iterations*
- 2. CAD / Simulations: 3D models (if applicable), Simulation results (if any)*

3. Fabrication & Integration Process Document

⚠ Guidelines

1. Show evolution of design (not just final result)
2. Explain why materials were chosen

Software Documentation

1. Software Overview

- What does the software do?

Micropython is the logical centre of this entire arrangement, handling the actual operation of all computational logics and corresponding instructions for the hardware. The code simultaneously manages instructions which are most important in keeping the drone in stable flight at all times, especially with 3 completely different flight states meant to always be on standby for each other.

MIT App Inventor creates an easier interface for tracking/inspecting the game, specifically for users and larger spectator audiences. It can get quite intense so as to playing the game itself to be keeping the score but the app takes over, offering an overall better play experience.

2. System Logic

Explain flow in simple steps

1. User switches drone on
2. User taps on esp32 in bluetooth devices list
3. The app screen shows "connected" (the esp32 and app are now connected via Bluetooth)
4. User presses "play" button
5. App sends confirmation to start game flight loop
6. esp32 receives data and engages upward thrust in motors towards the apex of parabola.
7. esp32 directs decreasing thrust toward one side, with ultrasound sensor ready for input.
8. Screen shows graphical path of drone
9. If ultrasound receives input - this is sent to esp32 to switch the direction of the parabola and continue the game

10. Data sent to the app, "hit!" displayed on screen
11. If no input is received, esp32 directs a timeout hover, and then resumes the game from the start.
12. Data sent to app, "score to: " (player #) is displayed. Score count is also present.
13. Score count stored in esp32.
14. Once a "set" (first to 5 points) ends, the parabola loop restarts.
15. Data sent to app: set count displayed in app.
16. Set count calculated via score count and stored in esp32.
17. When game, set, match condition is fulfilled (2:1 sets to both players): data sent to app
18. App declares "winner"
19. Esp32 runs "timeout thrust" for drone

3. Flowchart

Visual representation of logic

https://miro.com/app/board/uXjVHbwkr9g=/?share_link_id=412391022789

4. App Flow (If MIT App Inventor)

- Screen flow
- Button actions
- Data flow

https://miro.com/app/board/uXjVHbwkr9g=/?share_link_id=669102297637